

GRAPH THEORY STRATEGY & MENTAL MODELS

A Blueprint for Visualizing Templates, Fixing Core Mistakes, and Coding Flawlessly

1. THE GOLDEN RULE OF TRAVERSAL MECHANICS

CORE MISTAKE FIX Recursive Helpers Break Iterative Waves (The 11-June Rotting Oranges Incident)

Never create a recursive helper function when implementing **Breadth-First Search (BFS)**. Calling a function like `bfs(i, j - 1)` inside a recursive pattern forces the execution environment to use the system call stack. This converts your strategy into **Depth-First Search (DFS)**, driving execution down a single path rather than expanding in an even wave.

CORRECT TEMPLATE The Only Way to Write True BFS

BFS must be controlled entirely by an explicit queue structure and a single iterative loop. No recursive methods allowed.

```
while (queue.length > 0) {
  const current = queue.shift();
  // Safely process all 4 spatial neighbors or node edges iteratively right here
}
```

2. ENTRY EXECUTION STRATEGY

ARCHITECTURAL CHOICE When to use Global Nested Loops vs. Single Point Entry

- **Use Global Nested Loops Matrix Scan:** When the grid can contain *multiple, disconnected starting points or components* (e.g., finding independent islands in *Number of Islands*, or collecting all starting contamination zones in *Rotting Oranges*). You must scan every index to guarantee no sub-graph is skipped.
- **Use Single Point Entry:** Only when the problem statement explicitly restricts execution to a single designated coordinate boundaries or a specified root element (e.g., finding the shortest escape route starting strictly at top-left corner coordinate `(0, 0)`).

```
// For Multiple Disconnected States:
for (let i = 0; i < rows; i++) {
  for (let j = 0; j < cols; j++) {
    if (grid[i][j] === 1 && !visited[i][j]) dfs(i, j);
  }
}
```

3. DATA REPRESENTATION BLUEPRINT

MATRIX LAYOUT Working Directly inside the 2D Coordinate Grid

Keep operations direct and localized if node relations are defined strictly by spatial boundaries (Top, Bottom, Left, Right). Creating external maps here slows down execution and adds space complexity overhead.

```
const directions = [[0, 1], [0, -1], [1, 0], [-1, 0]]; // Pure spatial 4-way expansions
```

ADJACENCY MODEL When to Build an Adjacency List First

If dependencies are passed abstractly via structural relation pairs (e.g., course prerequisites [\[u, v\]](#), network routers, flight schedules), you must build an explicit Map model or Adjacency List immediately before processing logic.

```
const adjList = new Map();
for (const [course, prereq] of prerequisites) {
  if (!adjList.has(prereq)) adjList.set(prereq, []);
  adjList.get(prereq).push(course);
}
```

4. MULTI-SOURCE LEVEL SNAPSHOT PATTERN

TIME & STEPS STRATEGY Locking Level Queue Sizes

When tracking time, rounds, or minimum steps across multiple active starting states simultaneously, you must capture the queue's length as a static snapshot at the exact beginning of the loop level iteration. This prevents newly appended child elements from skewing your current frame's timing index.

```
while (queue.length > 0 && freshCount > 0) {
  let currentLevelSize = queue.length; // Lock size snapshot
  for (let i = 0; i < currentLevelSize; i++) {
    const [r, c] = queue.shift();
    // Infect neighbors, decrease fresh count, push new coordinates
  }
  minutes++; // Increment time only after processing the exact snapshot layer
}
```

"Lock down the template, master the visual layout, and run the loop with complete clarity."